Welcome to

instats

The session will begin shortly.

Start

A 3-DAY WORKSHOP ON LLMS FOR MEASUREMENT

Generative LLMs for wrangling and crawling.

Free-text extraction, polite crawling, and an end-to-end pipeline. Same reproducibility demands as Day 2. New tool.

INSTRUCTOR
Jihye Park · University of Geneva

PREREQUISITE
Day 2 hands-on cell finished running

DURATION
2 hours 30 minutes

DAY 2 MAKE-UP · 20 MINUTES

Fine-tuning, done right.

The live SetFit run did not cooperate yesterday. Here is the same idea on a model that works, taken straight from a paper of mine now in revision.

00

WHERE SETFIT SITS, AND WHY IT STALLED

Three rungs on the fine-tuning ladder.

RUNG 01 · DAY 1

Zero-shot

An off-the-shelf encoder with no training of your own. Cheap and reproducible. The ceiling is whatever the base model already knows.

RUNG 02 · YESTERDAY'S DEMO

SetFit, few-shot

A small contrastive head fit on a handful of labels. Powerful in principle. The live run depends on a fragile stack, and yesterday it would not converge in front of you.

RUNG 03 · TODAY'S MAKE-UP

Full fine-tune

Update the transformer weights on a few hundred hand-labeled examples. More setup. The validation numbers are the ones a reviewer will trust.

SetFit is not wrong. The demo simply hid the payoff behind a setup problem. The cleanest way to see what fine-tuning buys you is a fine-tune that has already shipped through peer review.

A LIVE EXAMPLE FROM MY OWN R&R

DeBERTa-v3, fine-tuned on trade-report text.

THE FRAMING

- The task is stance detection cast as natural-language inference.
- The **premise** is one paragraph from a US trade report on a country's IP regime.
- The **hypothesis** is a short standardized claim about that country.
- The model returns an entailment probability. I turn that into a signed stance and average within country and year.

Notebook on screen. `NTE_DeBERTa_V3_revised_colab.ipynb`, Part 1.

```
# premise "...the IPR situation in country X, year Y: <paragraph>" # hypothesis "The country has strong IP law." # model output entailment prob → signed stance on a -5 to +5 scale
```

How the model keeps track of word order.

TRADITIONAL BERT · ABSOLUTE POSITION

Numbered seats

BERT learns a fixed list of 512 numbered seats. The first word sits in seat 1, the next in seat 2, on down the line.

There is no seat 513, so BERT cannot read past 512 tokens. The wall is built in.

DEBERTA-V3 · RELATIVE POSITION

Distance between words

DeBERTa tracks how far apart two words are rather than which seat each one holds. "Three words to the left" means the same thing anywhere.

Distance still makes sense past word 512, so the model is not boxed in by a fixed chart.

Relative position embedding, in one line. The model encodes the gap between two words instead of the fixed slot each word sits in. That single change lets DeBERTa stretch past the 512-token wall that hard-stops BERT. The shipped deberta-v3-large is itself trained at 512 tokens, and because the positions are relative, people run it on longer passages toward 1024 and a little beyond. BERT cannot go there at all.

Sources. He et al. 2020, DeBERTa, arXiv:2006.03654. Shaw, Uszkoreit, and Vaswani 2018, relative position representations, NAACL. Devlin et al. 2019, BERT, NAACL.

WHAT THE FINE-TUNE ACTUALLY TRAINS ON

Thirteen hypotheses, three thousand pairs.

LABELS

300 paragraphs

Hand-labeled IP paragraphs, each paired with every hypothesis. Up to 3,900 premise and hypothesis pairs.

TARGETS

13 hypotheses

Trained on all of them at once, so the model learns factual markers and interpretive claims together.

HYPERPARAMETERS

Conservative

Learning rate 2e-5. Weight decay 0.03. Batch size 8. Small steps guard against overfitting a tiny corpus.

OUTPUT

One proxy

A weighted sum of entailment probabilities, scaled to a country-year IP evaluation score.

Part 2 of the notebook trains the production model on all 300 paragraphs. Part 3 scores 1,432 paragraphs across the full corpus.

EVERY DIAL WE SET DURING TRAINING, IN PLAIN TERMS

The training arguments, one by one.

ARGUMENT (OUR VALUE)	WHAT IT CONTROLS	THE TRADE-OFF
Learning rate `2e-5` start 2e-5, range 1e-5 to 5e-5	How big a step the model takes each time it corrects a mistake.	Too high overshoots and never settles. Too low crawls. A small value is the safe choice on little data.
Epochs start 3, range 2 to 4	How many times the model reads through the whole training set.	Too many and it memorizes the examples (overfitting). Too few and the pattern is not learned yet.
Batch size `8` start 16 to 32	How many examples it studies before each correction.	Larger is smoother and faster but uses more memory. Smaller fits a big model on one GPU.
Weight decay `0.03` start 0.01 to 0.1	A gentle pull that keeps the model's internal numbers from growing large.	Higher guards harder against overfitting but can blur real signal. Zero removes the brake.
Warmup start 6 to 10% of steps	Starting with tiny steps and easing the rate up over the first batches.	Skipping it risks wild swings early, when the fresh weights are most fragile.
Mixed precision `bf16` on when the GPU allows	Training in 16-bit numbers rather than 32-bit, with a 32-bit master copy kept for safety.	Halves memory and speeds training. Plain fp16 can throw NaN errors, so bf16 is the sturdier choice on newer GPUs.

All set once in `TrainingArguments`. The starting values follow the BERT and RoBERTa fine-tuning recipes (Devlin et al. 2019, arXiv:1810.04805, and Liu et al. 2019, arXiv:1907.11692). On a few hundred paragraphs, the small learning rate and the non-zero weight decay matter most.

STRATIFIED 10-FOLD CROSS-VALIDATION, FULL PIPELINE INSIDE EACH FOLD

The numbers a reviewer will actually read.

Accuracy

77.3%

Out-of-sample, dichotomized against hand-coded labels across 300 paragraphs.

Weighted F1

0.787

Binary split. Pooled across all 13 hypotheses the weighted F1 rises to 0.870.

Score correlation

0.68

Pearson r between predicted and hand-coded scores. Spearman rho is 0.689.

Factual vs. interpretive

.99 / .69

Factual hypotheses score near 0.9 to 0.99. Interpretive ones land lower, exactly as expected.

> The whole preprocessing pipeline runs **inside each fold**, so nothing from the held-out paragraphs leaks into training. That is the honest-validation rule from Grimmer, Roberts, and Stewart.

THE TAKEAWAY, THEN ON TO DAY 3

When the fine-tune is worth it.

FINE-TUNE WHEN

- Zero-shot has hit its ceiling on your task.
- You can afford a few hundred hand-labeled examples.
- Your construct is multidimensional, like IP protection across enforcement and legislation.
- You need a defensible, deterministic measurement for a paper.

SKIP IT WHEN

- A zero-shot encoder already clears your bar.
- You have almost no labels and no time to make them.
- The job is free-text wrangling rather than fixed-label classification. That is today.

Day 3 leaves the encoder behind and picks up the generative family for the wrangling jobs a fine-tune cannot reach.

TODAY

Four movements, from one prompt to a pipeline.

MOVEMENT 01 · 17 MIN

Why generative

The decision rule. When the encoder runs out of road and the generative tool becomes the right one.

MOVEMENT 02 · 35 MIN

Structured extraction

Fifty news leads, four JSON fields, prompt caching, and validation that does not need any hand-coding.

MOVEMENT 03 · 35 MIN

Lightweight crawling

Robots.txt live on screen. A polite Python fetcher. Raw HTML stored for reproducibility.

MOVEMENT 04 · 25 MIN

End-to-end pipeline

Crawl, parse, extract, validate, save. Run together on five fresh articles. One full row of output.

One ten-minute break after Movement 2. Closing reproducibility checklist in the last fifteen minutes.

WHERE WE HAVE BEEN

Days 1 and 2 in one slide.

DAY 01

The model zoo

Two families of LLMs. Encoders read text and return labels. Generative models continue text. Day 1 ran zero-shot DeBERTa on five paragraphs of US trade-policy text.

DAY 02

From a model to a measurement

Codebook, thirty-paragraph gold set, thirteen-hypothesis ensemble, country-year proxy plotted against external events, and a validation appendix.

DAY 03

The other family

Today we leave the encoder behind. We pick up the generative model and use it for tasks the encoder cannot easily do.

The encoder is the right tool for closed labels. There are tasks for which it is not. Today is about those tasks.

Three jobs the encoder cannot easily do.

JOB 01

Free-text extraction

"France and Kenya signed a bilateral health-sector aid agreement worth EUR 80 million."

Encoder can tell you it mentions a signing. It cannot tell you who signed, or for how much.

JOB 02

Normalization

"USA," "United States," "U.S.," "the United States of America."

Encoder labels each. Generative resolves them all to a single canonical form in one prompt.

JOB 03

Light summarization

A 600-word press release with three buried facts about the agreement.

Encoder needs a fixed schema of labels in advance. Generative picks out the three facts on first read.

Same reproducibility demands as yesterday. New tool. The intuition carries over.

FIRST MOVEMENT · 17 MINUTES

Why reach for generative.

The decision rule that tells you when the encoder runs out of road.

01

THE THREE JOBS, WITH THE DIAGNOSTIC QUESTION

One question tells you which family fits.

DIAGNOSTIC 01

Closed list?

Can the answer be expressed as a value from a finite list you wrote down before reading any text?

Yes. Encoder.
No. Generative.

DIAGNOSTIC 02

Messy input?

Is your input messy in a way an encoder hypothesis cannot fix? Country-name variants, date formats, alias resolution.

Yes. Generative.

DIAGNOSTIC 03

Variable schema?

Do you need three or four pieces of information that may or may not appear, with different surface forms each time?

Yes. Generative.

If two of three are yes, the generative tool is almost certainly the right starting point. If two of three are no, run an encoder first and reach for a generative model only if the encoder ceiling is hit.

THREE REAL COSTS THAT COME WITH THE NEW TOOL

Tradeoffs that go in your replication appendix.

COST 01

Per-token billing

Generative API calls are billed per input and output token. Today's demo on Anthropic Haiku 4.5 costs roughly five US cents for fifty leads plus five end-to-end articles.

Fine for a paper. Becomes a research-design constraint at million-paragraph corpus scale.

COST 02

Non-determinism

Even at temperature zero, two calls with the same prompt can produce slightly different outputs. Providers also retire models behind the scenes.

Pin model version. Run the determinism cell. Report the number.

COST 03

Prompt versioning

The prompt is a new artifact in your replication package. If the prompt changes, the output changes. Same for the model version.

Ship system prompt verbatim. Include date finalized. Hash it for the audit log.

None of the three costs existed in yesterday's encoder appendix. All three are now part of the methods burden.

When to reach for which family.

	CLOSED LABELS	FREE-FORM, MESSY, OR VARIABLE
Large corpus tens of thousands+	**ENCODER, FINE-TUNED** Yesterday's pipeline. Hand-coded gold, weighted ensemble. Determinism by construction.	**GENERATIVE + ENCODER HYBRID** Generative on upstream extraction. Encoder on downstream classification. Both validation tracks.
Small corpus hundreds to a few thousand	**ENCODER, ZERO-SHOT** Day 1's pipeline. NLI hypotheses. Cheap, deterministic. Often enough on its own.	**GENERATIVE, PROMPT-ONLY** Today's pipeline. Schema, system prompt, validators, audit log. Iteration speed for the win.

Most political-science measurement projects sit in the bottom-left cell. Today is about the right column. Hybrid pipelines combine them.

SECOND MOVEMENT · 35 MINUTES

Structured extraction.

Fifty news leads. Four JSON fields. The single most common pattern political scientists need from a generative model.

02

How we actually call the model.

AN API CALL, IN PLAIN TERMS

Rather than typing into a chat box, your notebook sends the request for you and gets the answer back as data. A few lines do the whole job.

```
# the whole call
resp = client.messages.create(
    model="claude-haiku-4-5-...",
    system=SCHEMA_PROMPT,    # fixed, cached
    messages=[{"role": "user",
               "content": lead}],
    temperature=0,
)
```

The live demo runs exactly this, fifty times, in a loop.

WHY NOT JUST USE THE CHAT APP

- **Scale.** Code loops over fifty or fifty thousand documents unattended. The chat window takes one message at a time, by hand.
- **Reproducibility.** The call records the model version, the prompt, and the settings, so you can re-run and ship it. The chat window keeps no record and hides its system prompt.
- **Control.** You set temperature, max tokens, caching, and the model pin. The chat app keeps those dials out of reach.
- **Clean output.** You get JSON straight into your table instead of prose to copy out of a window.

> **The line to remember.** The chat window is for exploring an idea. The API is for measurement, since a result you cannot re-run is hard to defend in a methods appendix.

Decide the schema before you touch any code.

```
{ "signatories": list[str], # parties to the agreement
"agreement_type": enum, # closed taxonomy, 12 values
"year": int | null, # 1990-2030 or null "sector":
enum, # closed taxonomy, 13 values }
```

THREE PROPERTIES

- **Finite.** The field set is decided in advance. Adding a new field after the fact means re-running every call.
- **Typed.** Every field has a type and, where possible, a closed taxonomy. The validator in section 5 of the notebook reads off the type signatures.
- **Pinned.** The schema is documented in the system prompt and in the replication package. Future-you needs both.

TWO PROMPTS, ONE EXTRACTION

System prompt holds the schema. User prompt holds the lead.

SYSTEM PROMPT · CACHED · SAME EVERY CALL

```
You are an information extraction system. Extract a JSON object with exactly four
fields.

  "signatories": list of strings, normalized to short country names.
  "agreement_type": one of [bilateral_trade, multilateral_trade, climate,
defense, health, financial_assistance, IP, security_cooperation, infrastructure,
technology, other].
  "year": four-digit integer or null.
  "sector": one of [trade, climate, defense, health, finance, IP, security,
infrastructure, technology, energy, agriculture, other].

Return ONLY the JSON. No markdown.

Example output:
{"signatories": ["France", "Kenya"], "agreement_type": "health", "year": 2024,
"sector": "health"}
```

Cached at ~10% of base rate across 50 calls. The example is the highest-leverage line.

USER PROMPT · PER ROW · NOT CACHED

```
"On 8 September 2020, the United Arab Emirates and Israel signed
the Abraham Accords, normalizing diplomatic relations and
establishing cooperation in security, technology, and tourism."
```

One news lead per call. Variable. Not cacheable.

[DEMO] · LIVE

Switch to `demo.ipynb` sections 1 to 4.

Run together

Section 1. Mount Drive. Install Anthropic SDK. Pull `ANTHROPIC_API_KEY` from `userdata`.

Section 2. Load `news_leads.csv`. Inspect three sample leads.

Section 3. Define the schema. Print the system prompt and the closed taxonomies.

Section 4. Run extraction on a single lead. Inspect the parsed JSON, input and output token counts, and any cache-related counters in the response.

If the API call returns a 401, double-check that the key is saved under the key icon in Colab and that the cell pulled it via `userdata.get()`.

Four checks you get for free without writing any gold.

CHECK 01

Schema validation

Every output parses as JSON. Every required field is present. Every value matches its declared type or taxonomy. Programmatic.

CHECK 02

Determinism

Same prompt, two runs, temperature zero. Count identical outputs. The drift you see is the floor on reproducibility.

CHECK 03

Sample audit

Read five random outputs by hand. Ten minutes of human time. Smallest defensible spot-check.

CHECK 04

Audit log

One row per call. Lead, prompt hash, timestamp, response, token counts, cost. The replication artifact.

Necessary, but not sufficient. These four checks are prerequisites for any further validation. They do not replace a hand-coded gold set. Slide 28 is where the gold set comes back as the missing piece for a publishable paper.

What you should expect to see when the cells run.

Schema pass rate

~98%

50 leads, four-field schema with one example. Modern Haiku-class models clear the high-90s on first pass.

Determinism

~9 / 10

10-lead subset, two runs, temperature 0. The drift comes from leads with ambiguous sector tagging.

Audit rows

50

One per call. System-prompt hash identical across all 50 because the prompt is cached.

Estimated cost

~$0.01

50 calls at Haiku 4.5 pricing with prompt caching. Well within the few-cents per replication budget.

Numbers are illustrative based on typical runs. The exact values from the live demo will appear in the notebook output. If Anthropic ships a model update between sessions, expect the determinism number to shift before you re-pin.

From a schema to an audit log in five notebook sections.

WHAT YOU BUILT

- A four-field schema with closed taxonomies pinned in the system prompt.
- A two-part prompt with a stable system message and a per-row user message.
- Fifty extraction calls with prompt caching at ~10% of base input rate.
- Programmatic schema validators and a determinism check.
- A five-record sample audit and a full audit log with cost.

WHAT IS STILL MISSING

- A hand-coded gold set with field-level F1 against truth.
- A subgroup breakdown of pass rate. Sector, year, signatory count.
- A simpler-baseline comparison. Rule-based extraction or a smaller model.
- A cost projection at production scale.

Slide 28 brings these back. Today's pipeline is the template you reuse when you have the budget for one.

Stop